

INTRODUCTION TO .NET FRAMEWORK

Learning Objectives

- 1.1 INTRODUCTION
 - 1.2 INTRODUCTION TO .NET FRAMEWORK
 - 1.3 ORIGINS/GENESIS OF .NET TECHNOLOGY
 - 1.4 FEATURES OF .NET
 - 1.5 .NET BINARIES
 - 1.6 ARCHITECTURE OF .NET FRAMEWORK
 - 1.6.1 Common Language Runtime (CLR)
 - 1.6.2 .Net Framework class library
 - 1.7 .NET VERSIONS
 - 1.8 MICROSOFT INTERMEDIATE LANGUAGE
 - 1.9 META DATA
 - 1.9.1 Benefits/Uses of Metadata
 - 1.10 .NET TYPES
 - 1.11 .NET NAMESPACES
 - 1.12 COMMON LANGUAGE RUNTIME
 - 1.13 COMMON TYPE SYSTEM
 - 1.14 COMMON LANGUAGE SPECIFICATIONS
 - 1.15 .NET APPLICATIONS USING COMMAND LINE COMPILER AND VISUAL STUDIO
 - .NET IDE
- REVIEW QUESTIONS

1.1 INTRODUCTION

More than 20 years ago, Tim Berners-Lee performed the first transmission across HTTP (Hypertext Transfer Protocol). Since then, HTTP has become exponentially more popular, expanding beyond a small group of computer-science visionaries to the personal and business sectors. Today, it's almost a household word.

When HTTP was first established, developers faced the challenge of designing applications that could discover and interact with each other. To help meet these challenges, standards such

as HTML (Hypertext Markup Language) and XML (Extensible Markup Language) were created. HTML established a simple language that can describe how to display rich documents on virtually any computer platform. XML created a set of rules for building platform-neutral data formats that different applications can use to exchange information. These standards guaranteed that the Web could be used by anyone, located anywhere, using any type of computing system.

At the same time, software vendors faced their own challenges. Not only did they need to develop languages and programming tools that could integrate with the Web, but they also needed to build entire frameworks that would allow developers to architect, develop, and deploy these applications easily. Major software vendors including IBM, Sun Microsystems, and Microsoft rushed to meet this need with a host of products.

With .NET, Microsoft has created an integrated suite of components that combines the building blocks of the Web—mark-up languages and HTTP—with proven object-oriented methodology.

1.2 INTRODUCTION TO .NET FRAMEWORK

Microsoft .NET is not a programming language, it is a framework that supports programming languages such as Microsoft Visual C#, Microsoft Visual Basic.NET, Microsoft Visual J#.NET and Microsoft Visual C++.NET. These are also called as Compliant Languages of .NET Framework. It provides a powerful Integrated Development Environment (IDE) that Supports drag and drop feature.

Microsoft .NET enables you to develop applications for different environments and devices. For example, you can build Web applications for a highly distributed environment such as the Internet. You can also create traditional Windows-based applications, server components, and applications that can run on any device, such as PC or a mobile device. Web-based applications can allow seamless access by users from any where in the world and from any type of client machines.

You can apply the open standards such as XML (eXtensibleMarkup Language), SOAP (Simple Object Access Protocol), and HTTP (Hyper Text Transfer Protocol) for developing your web-applications and web applications created with such open standards can interact well with other applications of other platforms.

Further, you can use .NET Framework for running your applications in almost all platforms such as Linux, Macintosh and UNIX without any hitches. The presence of many off-the-shelf libraries in .NET Framework can assist you in developing your applications in a faster, cheaper and easier manner.

The functionality of .Net Framework supporting many programming languages is due to the use of the powerful the Common Language Runtime (CLR) engine. The application programming codes are first compiled by the language compiler into a Microsoft Intermediate Language (MSIL) or Intermediate Language(IL) code instead of native codes and the MSIL (which is nothing but an instruction set) is in turn compiled by CLR at runtime and converted into the native code for running the application.

The platform independent feature of .NET Framework can be attributed to CLR and the same CLR also takes care of run-time services such as memory processes, security enforcement, integration of language, and thread management. Hence, you can make use of the various infrastructures that have been provided in .NET Framework for creating your web-applications.

.NET overcomes one of the biggest challenges of the software industry by providing language interoperability feature which means each language can use code written in other languages.

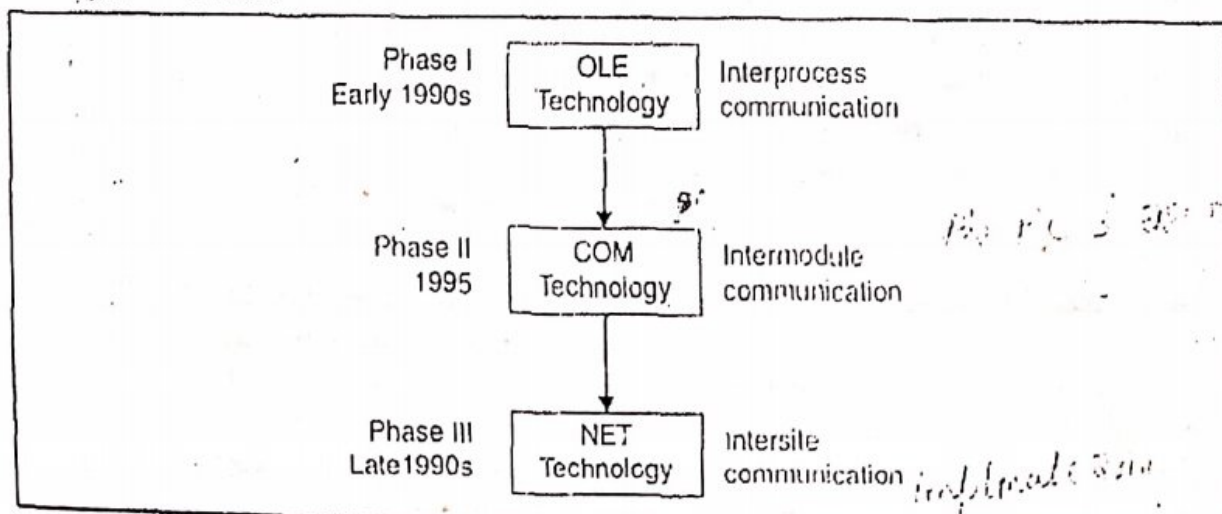
The major facts about .net are:

- .NET is a new Internet and web technology strategy by Microsoft.
- .NET should not be considered as a type of operating systems; it is a set of technologies that provides a completely new internet and web based infrastructure.
- Software delivered by .NET is called as Web Services or Web Applications and most of the services provided by .NET are also called as Universal Services.
- .NET is completely based on new www standards and it runs on any browser independent of the platform.

1.3 ORIGINS/GENESIS OF .NET TECHNOLOGY

Before going into the details of .NET features further, we shall see how the concept of .NET was evolved over the past ten years. The current technology of .NET has gone through three significant phases of development:

- OLE Technology
- COM Technology
- .NET Technology



OLE Technology

OLE (Object Linking and Embedding) is Microsoft's component document technology. It allows us to link elements from different applications with each other. With OLE, we can dynamically link files and applications together. An object is a combination of data and the

application needed to modify that data. We can embed objects in or link them to documents created with a different application.

For instance, an Excel spreadsheet can be embedded within a Microsoft Word document using OLE. Double-clicking the embedded spreadsheet will launch Excel and allow the user to modify the sheet.

COM Technology

Microsoft COM (Common Object Model) technology in the Microsoft Windows-family of Operating Systems enables software components to communicate. COM is used by developers to create re-usable software components, link components together to build applications, and take advantage of Windows services. COM objects can be created with a variety of programming languages. Microsoft provides COM interfaces for many Windows application programming interfaces such as Direct Show, Media Foundation, Packaging API, Windows Animation Manager, Windows Portable Devices, and Microsoft Active Directory (AD).

COM is used in applications such as the Microsoft Office Family of products. For example COM OLE technology allows Word documents to dynamically link to data in Excel spreadsheets and COM Automation allows users to build scripts in their applications to perform repetitive tasks or control one application from another. COM+ is the name of the COM-based services and technologies first released in Windows 2000. COM+ automatically handles programming tasks such as resource pooling, disconnected applications, event publication and subscription and distributed transactions.

.NET Technology

.Net is technology of collection or set of technologies to develop windows and web applications. .Net technology is developed by Microsoft and launched in feb. 2002, by basic definition, a Microsoft's new Internet Strategy. It was originally called as NGWS (Next Generation Web Services). It is considered to be one of the most popular, powerful and useful Internet technology available today. Today, the term used in replacement of NGWS is .NET because NGWS was originally used to describe Microsoft's plans for developing Internet Based Platform for the next generation windows services. It included various changes in the programming models being used before the introduction of .NET and it brought many changes in the user interfaces, xml schemas, file systems and application integration models.

According to Steve Ballmer who quoted in January 2000, "Delivering an Internet-based platform of Next Generation Windows Services is the top priority of our company. The breakthroughs we're talking about here include changes to the programming model, to the user interface, to the application integration model, the file system, new XML schema...."

Some people even considered Microsoft .Net as a new operating system; which is not the case. Microsoft .Net is a Web and Internet Strategy introduced by Microsoft. .Net introduces improved programming models and provide new and better Web based infrastructure. One of the key facts about .Net is that it delivers software as web services and the framework provided by Microsoft .Net is for universal services. Another important fact about .Net that you should always remember is that it can run on any browser and on any platform. Microsoft .Net is purely based

on new web standards and is based on a new model; server centric computing model. Here are some of the main web standards on which .Net is based:

- **UDDI:** is a standard on which .Net is based in order to discover & search various Web Services. Remember that in Microsoft's .Net terminology, Software is called as web services.
- **SOAP:** defines the standard format to be used when requesting any Web Service.
- **HTTP:** the classic standard that defines the communication protocol between various Internet Applications regardless of their platform.
- **XML:** defines the format to be used for exchanging data between various Internet applications.

1.4 FEATURES OF .NET

.NET is the framework for which we develop applications. It sits in between our application programs and operating system. Applications developed for .NET run inside .NET and are controlled by .NET. It supports both Windows and web applications.

.NET provides an object oriented environment. It ensures safe execution of the code by performing required runtime validations. For example, it is never possible to access an element of an array outside the boundary. Similarly, it is not possible to a program to write into another programs area. The runtime validations performed by .NET makes the entire environment robust.

Talking about the principal design features of .NET framework, one of its main features is that it provides developers with the freedom of using similar skill set to develop different types of computer application which can be used across various types of devices as well as environments. The next of the principal design feature of Microsoft .NET framework is that it is used on those computers systems which run on Microsoft windows operating systems. There is a huge library consisting of coded solutions presented for simple programming issues and a virtual machine to direct the execution of those programs which were written specially for the framework.

It is important to note down regarding the features of .NET framework is that its library can be used by all the programming languages which are supported by .NET facilitates. Presented below is the summary of Microsoft .NET framework features, which will help you gain better understanding:

1. Language Interoperability

Language interoperability means code written in a language can be used by other languages and there is an access provided to COM components in the System. In other words, this is the ability of code, to interact with code, which is written using a different programming language. Language Interoperability can help to maximize code reusability and, therefore, improve the efficiency of the development process. This can be achieved because all .NET code is compiled to Intermediate Language (IL) whether you are developing in C#, VB.NET, J#.NET or

any other .NET language. This means it is possible to build applications with modules written in different languages, because when the application is compiled it will all be compiled to a common language, IL. This means a class written in one language can inherit from a class written in another language, or an object can directly call a method of another class written in another .NET language.

2. Common Language Runtime

It is actually a virtual machine component of the .NET framework. All the programs run under the direction of the CLR, thus ensuring some specific properties and behaviours in the areas of memory management, security, and exception handling.

The Common Language Runtime consists of components that

- Loads the IL code of a program into the runtime,
- Compile the IL code into native code,
- Execute and manage the code,
- Enforce security and type safety, and provide thread support and other useful services.

The code that runs within the CLR is called as managed code, whereas the code that runs outside the CLR is called unmanaged code. The CLR also provides Interoperability layer, which allows both the managed and unmanaged code to interoperate with each other.

It works as a layer between Operating Systems and the applications written in .Net languages that conforms to the Common Language Specification (CLS). The main function of Common Language Runtime (CLR) is to convert the Managed Code into native code and then execute the program. The Managed Code compiled only when it needed, that is it converts the appropriate instructions when each function is called. The Common Language Runtime (CLR)'s Just In Time (JIT) compilation converts Intermediate Language (MSIL) to native code on demand at application run time.

Language Independence

Language Independence makes use of the power of various languages in one platform. It means programmer's can develop their applications in any of the language supported by .NET framework. .NET framework supports various programming languages such as Microsoft Visual Basic .NET, Microsoft Visual J#.NET and Microsoft Visual C++.NET.

After Compilation the source code of all these languages converted into Common Intermediate Language (CIL) that enhances Language Independence.

Base Class Library

The Base Class Library (BCL) is a library of functionality available to all languages using the .NET Framework. The BCL provides classes that encapsulate a number of common functions, including file reading and writing, graphic rendering, database interaction, XML document manipulation.

The following figure (Fig. 1.1) shows the .NET framework Base Class Library:

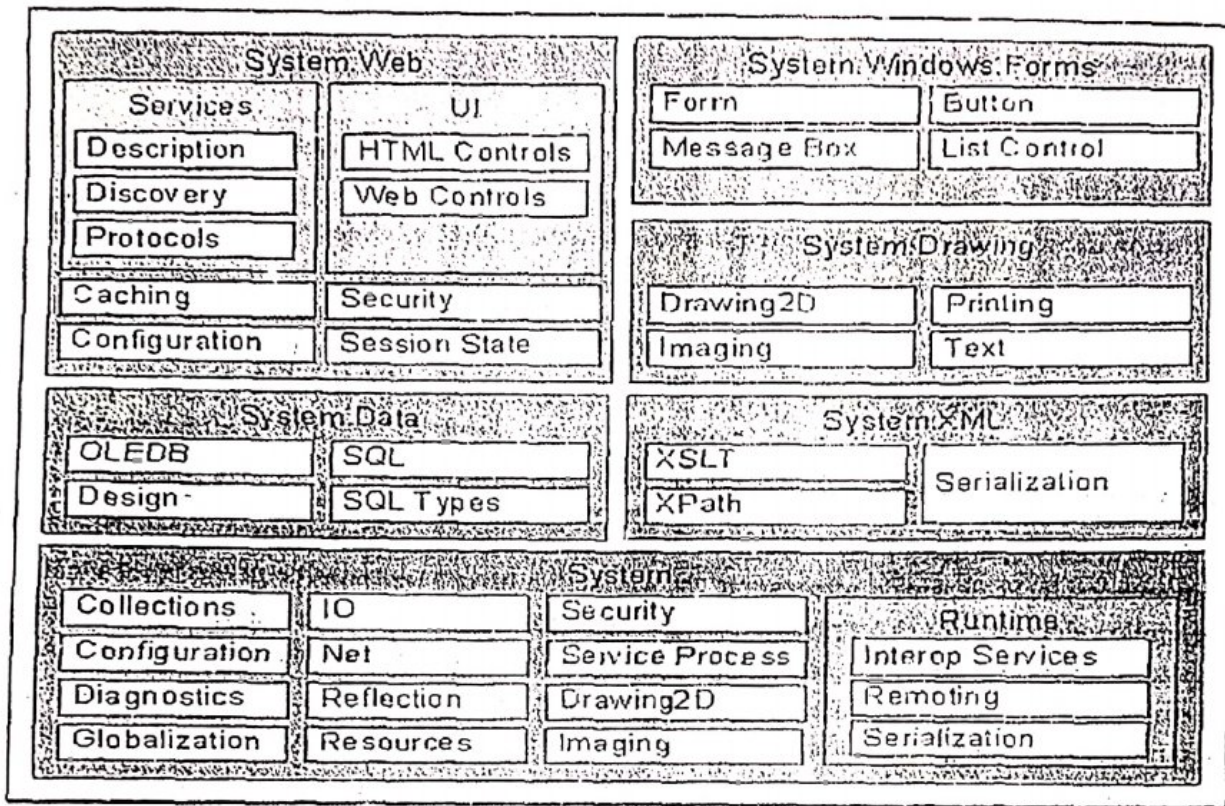


Fig. 1.1: Base class library

5. Automatic memory management (Garbage Collection)

While developing applications developers had to develop an eye on system resources like memory. Memory leaks were major reason in failure of applications. .NET takes this worry away from developer by handling memory on its own. The garbage collector takes care of freeing unused objects at appropriate intervals.

A major feature of .NET framework CLR is that the runtime automatically handles the allocation and release of an object's memory resources. Automatic memory management enhances code quality and developer productivity without negatively impacting expressiveness or performance. The Garbage Collector (GC) is responsible for collecting the objects no longer referenced by the application. The GC may automatically be invoked by the CLR or the application may explicitly invoke the GC by calling it. Objects are not released from memory until the GC is invoked.

6. Security

Windows platform was always criticized for poor security mechanisms. Microsoft has taken great efforts to make .NET platform safe and secure for enterprise applications. Features such as type safety, code access security, and role based authentication make overall application more robust and secure.

7. Easy Development of Web Applications

With the help of ASP.NET available on .NET, It become easy to develop web applications and

web services. ASP.NET provides an event driven programming model that simplify development of web pages with complex user interface. ASP.NET server controls provide advanced user interface elements such as calendar and grids that save lot of coding from programmer's side.

8. Type Safety

During program execution, the type checker ensures that all objects and values, and the references to those objects and values, have a valid type. For example, the type checker ensures that only an integer value is assigned to an integer variable. The type checker also ensures that only valid operations are performed on the objects or values. For Example, the type checker ensures that only addition, subtraction, division, and multiplication are performed on an integer value.

9. Managed Multithreading Support

Applications running in the CLR can utilize the managed multithreading support. The .NET Framework allows a single process to be divided into one or more sub processes called application domains. Each application domain can contain one or more threads. The runtime monitors all the threads that have executed code within its process. Multithreading requires careful programming. For most tasks, you can reduce complexity by queuing requests for execution by thread pool threads.

10. Multi-Device Support

Modern life style is increasingly embracing mobile and wireless devices such as PDAs, mobiles and handheld PCs. .NET provides promising platform for programming such devices. .NET Compact Framework and Mobile Internet Toolkit are step ahead in this direction.

1. Side by Side Execution

The .NET Framework allows you to deploy multiple versions of an application on a system by using assemblies. Assemblies are the deployment units in the .NET Framework. An assembly contains the IL code and metadata. The metadata contains information such as the version of the other assemblies on which the assembly depends. The Common Language Runtime uses the version information in the metadata to determine application dependencies and enables you to execute multiple versions of an application side-by-side.

In-process side-by-side hosting enables an application to load and activate multiple versions of the common language runtime (CLR) in the same process. For example, you can run applications that are based on the .NET Framework 2.0 SP1 and applications that are based on .NET Framework 4.0 in the same process. Older components continue to use the same CLR version, and new components use the new CLR version.

2. Common Type System

This provides the necessary data types, value and object types, which you need to develop applications in different languages. All the .NET languages share CTS. This implies that a String in Visual Basic .NET is the same as a String in Visual C# or in Visual C++ .NET. All the .NET languages have access to the same class libraries. In addition, all languages are equally

powerful. There is no superior language in .NET i.e. Visual Basic .NET is as powerful as Visual C# or Visual C++ .NET.

13. Parallel Computing

The .NET Framework 4.0 introduces a new programming model for writing multithreaded and asynchronous code that greatly simplifies the work of application and library developers. The new model enables developers to write efficient, fine-grained, and scalable parallel code in a natural idiom without having to work directly with threads or the thread pool. The new Parallel and Task classes, and other related types, support this new model. Parallel LINQ (PLINQ), which is a parallel implementation of LINQ to Objects, enables similar functionality through declarative syntax.

14. Dynamic Language Runtime

The dynamic language runtime (DLR) is a new runtime environment that adds a set of services for dynamic languages to the CLR. The DLR makes it easier to develop dynamic languages to run on the .NET Framework and to add dynamic features to statically typed languages. To support the DLR, the new System.Dynamic namespace is added to the .NET Framework. In addition, several new classes that support the .NET Framework infrastructure are added to the System.Runtime.CompilerServices namespace.

15. Distributed Architecture

Today's distributed applications are much different than those we will see in the future. Microsoft certainly believes this; they say they are betting the company on the concept of distributed Web services.

For example, today when a user is interacting with a portal site, it appears to them that they are working with one remote server. Most of us know that is normally not the case, at least for a site of any significant size. There are various servers and applications behind the scenes accessing information on several remote sites, combining it with information from their user database and merging it all into an integrated product that is delivered to the user via their browser. As useful as these types of applications are, they are all very complex to develop and maintain. Each provider of information has developed different interfaces to access data and processes on their servers. This redundant development is grossly inefficient and for the most part fairly boring, so there has been a great deal of activity around three standards to streamline the process: XML, SOAP, and UDDI. As we discussed earlier, these are used in .NET and also in competing, less well known initiatives from IBM and Sun.

1.5 .NET BINARIES

Regardless of which managed language you choose, all .NET-aware compilers emit binaries that share the same internal composition. Although .NET binaries share the same file extension as a classic COM server (*.dll and *.exe), they are completely different under the hood. First and foremost, .NET binaries do not contain platform-specific instructions, but rather platform-agnostic IL code that is compiled to platform-specific instructions using a just-in-time compiler.

In addition to the IL instruction set, .NET binaries contain full and complete metadata that describes each and every .NET type referenced within the binary. Finally, a .NET compiler emits binaries containing a *manifest* that describes the binary shell itself.

In addition to a unique internal fabric, .NET binaries have been given a new name, *assembly*. Assemblies are the unit of deployment and unit of versioning under the .NET platform. Specifically speaking, an assembly can be a single-file assembly or multifile assembly. Single-file assemblies are a single *.dll or *.exe file that contains all .NET types in a single unit. Multifile assemblies, on the other hand, are a collection of related files. When developers build multifile assemblies (using the command line compilers supplied with the .NET SDK), the end result is a collection of files that are versioned as a single unit.

.NET assemblies differ from classic COM servers in other ways as well. Perhaps the most marked difference is that .NET assemblies are *not* registered in the system registry.

.NET applications are deployed as assemblies, which can be a single executable or a collection of components. When you create a .NET application, you are actually creating an assembly, which contains a manifest that describes the assembly. This manifest data contains the assembly name, its versioning information, any assemblies referenced by this assembly and their versions, a listing of types in the assembly, security permissions, its product information (company, trademark, and so on), and any custom attribute. There are following Features of assemblies:

- (1) Assemblies are self-describing.
- (2) Version dependencies are recorded inside an assembly manifest.
- (3) Assemblies can be loaded side-by-side.
- (4) Application isolation is endured using application domains.
- (5) Installation can be as easy as copying the files that belong to an assembly.
- (6) Assemblies can be private or shared.

An assembly is the actual .dll file on your hard drive where the classes in the .NET Framework are stored. For example, all the classes contained in the ASP.NET Framework are located in an assembly named System.Web.dll. More accurately, an assembly is the primary unit of deployment, security, and version control in the .NET Framework. Because an assembly can span multiple files, an assembly is often referred to as a "logical" dll. There are two types of assemblies: private and shared.

Private and Shared Assemblies

A **private assembly** is found either in the same directory as the application, or within one of its subdirectories. With the private assembly it is not necessary to think about naming conflicts with other classes or versioning problems. Private assemblies are the normal way to build applications, especially when applications and components are built within the same company.

While using **shared assemblies**, we have to aware of some rules. The assembly must be unique and therefore must also have a unique name called a strong name. Shared assemblies are mostly used when a vendor, different from that of the application, builds the component.

1.6 ARCHITECTURE OF .NET FRAMEWORK

The structure of the .NET Framework is a layered architecture. Figure 1.2 shows a high-level representation of the .NET Framework architecture. The first level of the representation is the operating system; the .NET layer is located between the system and applications. The second level is the Common Language Runtime (CLR), which provides the part of the .NET Framework doing the most work. The next level is the Base Class Library (BCL), which provides all .NET objects that can be used both in your code and by Visual Basic when creating applications. The BCL also provides the infrastructure of several .NET technologies that you use in building applications, such as WPF, Windows Forms, ASP.NET, WCF, and so on. The last level is represented by applications that rely on the previous layers.

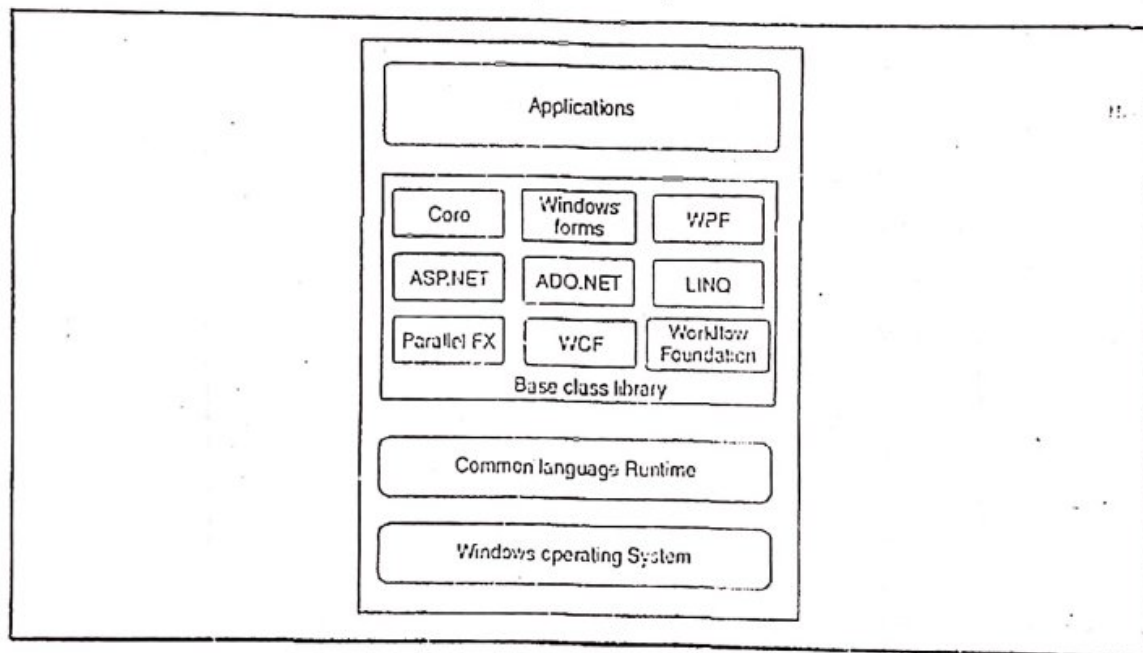


Fig.1.2

There are two main components of the .NET Framework as:

- Common Language Runtime.
- Net Framework class library.

1.6.1 Common Language Runtime (CLR)

The most important component of the .NET Framework is the Common Language Runtime (CLR). The CLR manages and executes code written in .NET languages and is the basis of the .NET architecture, similar to the Java Virtual Machine. The CLR activates objects, performs security checks on them, lays them out in memory, executes them, and garbage-collects them. The Common Language Runtime consists of components that

- Loads the Intermediate Language (IL) code of a program into the runtime.

- o Compile the IL code into native code,
- o Execute and manage the code,
- o Enforce security and type safety, and provide thread support and other useful services.

The code that runs within the CLR is called as managed code, whereas the code that runs outside the CLR is called unmanaged code. The CLR also provides Interoperability layer, which allows both the managed and unmanaged code to interoperate with each other.

Managed Code

The code, which is developed in .NET framework, is known as managed code. This code is directly executed by CLR with help of managed code execution. Any language that is written in .NET Framework is managed code.

Managed code uses CLR which in turns looks after your applications by managing memory, handling security, allowing cross - language debugging, and so on. The process of managed code is shown in fig. 1.2.

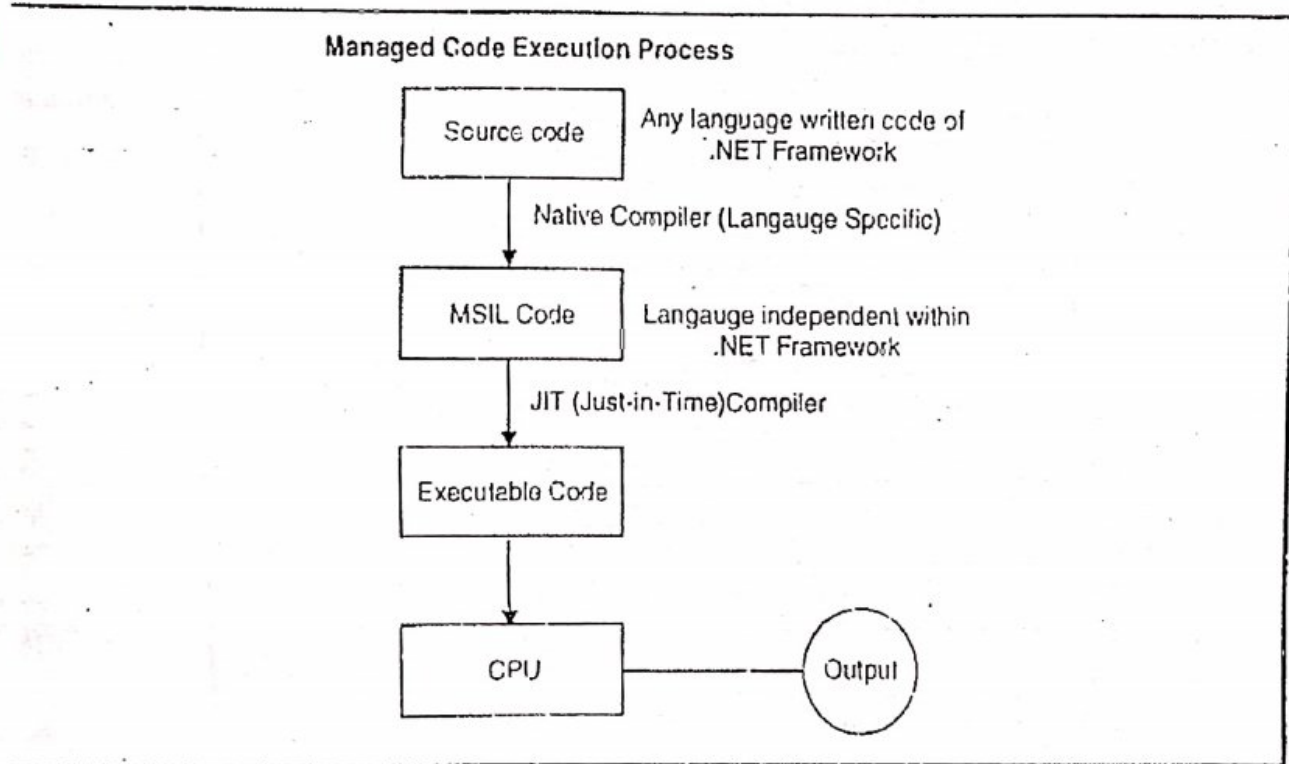


Fig. 1.3

Unmanaged Code

The code, which is developed outside .NET, Framework is known as unmanaged code. Applications that do not run under the control of the CLR are said to be unmanaged, and certain languages such as C++ can be used to write such applications, which, for example, access low - level functions of the operating system. Background compatibility with code of 32-bit, ASP and COM are examples of unmanaged code. Unmanaged code is executed with help of wrapper classes. The unmanaged code process is shown in Fig. 1.4.

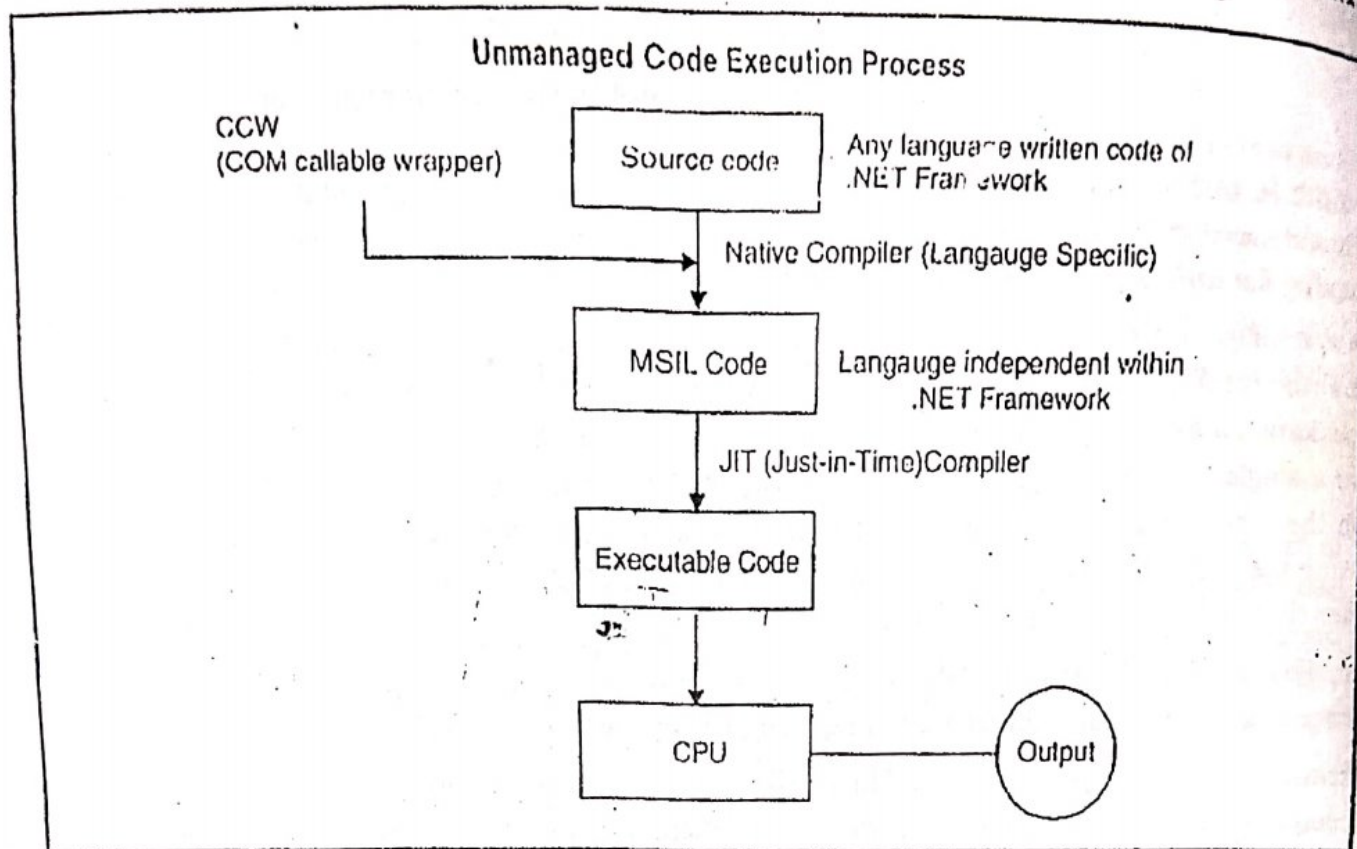


Fig. 1.4

1.6.2 .Net Framework class library

The .Net Framework class library (FCL) provides the core functionality of .Net Framework architecture. The .Net Framework Class Library (FCL) includes a huge collection of reusable classes, interfaces, and value types that optimize the development process and provide access to system functionality.

The .Net Framework class library (FCL) organized in a hierarchical tree structure and it is divided into Namespaces. Namespaces is a logical grouping of types for the purpose of identification. Framework class library (FCL) provides the consistent base types that are used across all .NET enabled languages. The Classes are accessed by namespaces, which reside within Assemblies. The System Namespace is the root for types in the .NET Framework. The .Net Framework class library (FCL) classes are managed classes that provide access to System Services. The .Net Framework class library (FCL) classes are object oriented and easy to use in program developments. Moreover, third-party components can integrate with the classes in the .NET Framework.

The .NET Framework class library provides the types that are common to all .NET languages. Programmers can use these types to develop different kinds of applications such as:

- o Console Applications
- o Windows and Web Forms
- o XML Web Services

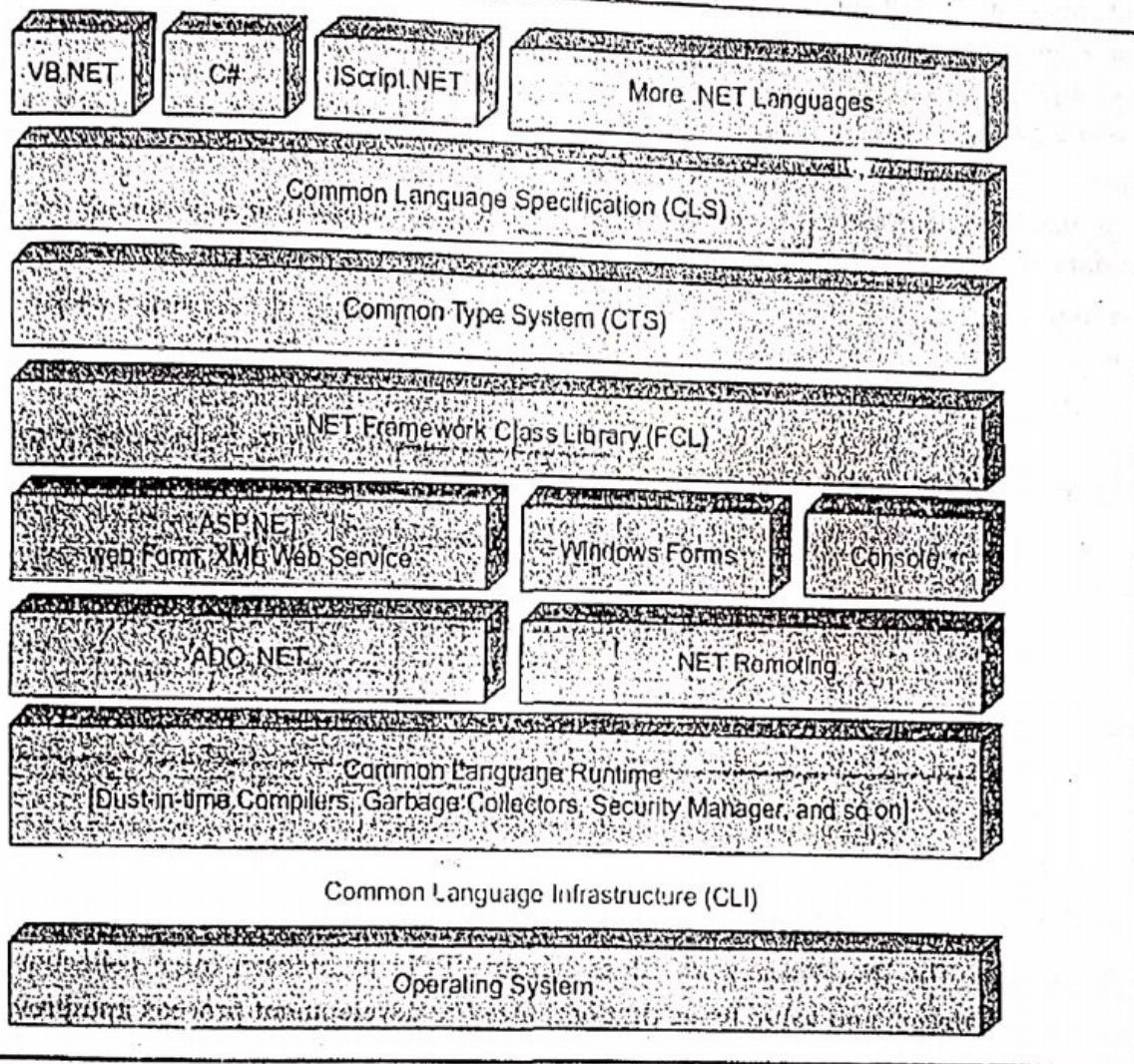


Fig. 1.5: Detailed .Net Architecture

With the .Net Framework you can make your codes written in fewer lines and other favorable features such as easy web settings, easy deployment of applications, easy compilation procedures, easy Web configuration makes the .NET Framework a great platform to work with.

7. .NET VERSIONS

Microsoft started development on the .NET Framework in the late 1990s originally under the name of Next Generation Windows Services (NGWS). By late 2000 the first beta versions of .NET was released. Fig 1.6 summarizes the version history:

Each version of the .NET Framework contains the common language runtime (CLR) as its core component, and includes additional components such as the base class libraries and other managed libraries.

Each new version of the .NET Framework retains features from the previous versions and adds new features. Although the CLR is the core component of the .NET Framework, the CLR is identified by its own version number apart from the .NET Framework version number. Some

versions of the .NET Framework include a new version of the CLR, but others use an earlier version.

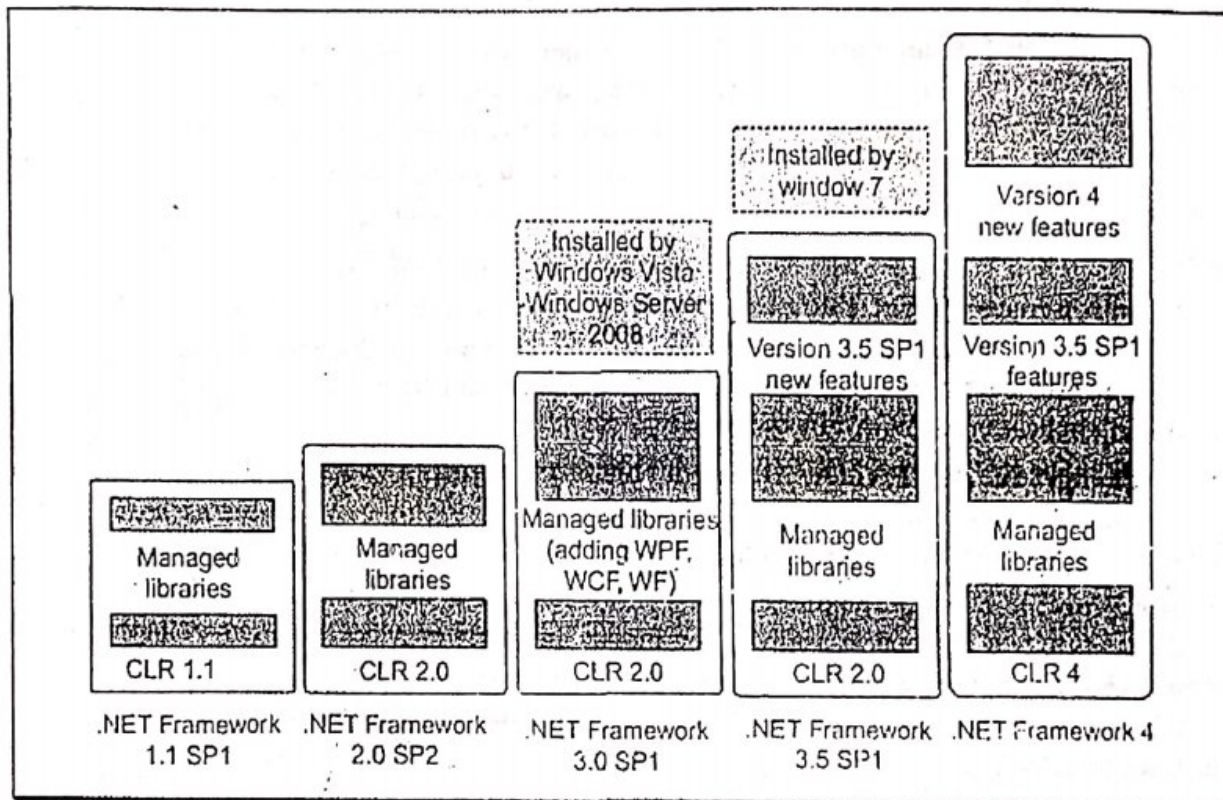


Fig. 1.6

For example, the .NET Framework version 4 includes CLR version 4, but the .NET Framework 3.5 includes CLR 2.0. The version of the CLR on which an application is running can be determined by retrieving the value of the Environment version property.

You do not have to install previous versions of the .NET Framework or the CLR before you install the latest version, each version provides the necessary components.

The following table (Table 1) provides a brief review of the .NET Framework versions and the associated CLR version. It also shows the Visual Studio version that provided the development environment when that version of the .NET Framework was introduced.

Table 1: Net framework versions

.NET Framework Version	CLR Version	Visual Studio Version
1.0	1.0	Visual Studio .NET
1.1	1.1	Visual Studio .NET 2003
2.0	2.0	Visual Studio 2005
3.0	2.0	Visual Studio 2005
3.5	2.0	Visual Studio 2008
4	4	Visual Studio 2010

All these .NET Framework Versions can be described as follows:

(a) .NET Framework 1.0

This is the first release of the .NET Framework. It is supported by Windows NT 4.0, Windows 98, Windows 2000, Windows ME, Windows XP. Developers using the .NET Framework version 1.0 must download the .NET Framework data provider for ODBC and for Oracle from <http://msdn.microsoft.com/downloads>.

(b) .NET Framework 1.1

This is the first major .NET Framework upgrade. This version was subsequently updated twice, with Service Pack 1 (SP1) and SP2. It is supported by Windows NT 4.0, Windows 98, Windows 2000, Windows ME, Windows XP. If .NET Framework 1.0 is removed, installing only .NET Framework 1.1 also provides the system support for version 1.0, except in rare instances where an application will not run because it checks the version number of a library.

• Changes in .NET Framework 1.1 in comparison with .NET Framework 1.0

- o Built-in support for Mobile ASP.NET Controls. Previously available as an add-on for .NET Framework, now part of the framework. The .NET Framework version 1.1, incorporates the mobile controls into the .NET Framework and Visual Studio .NET distributions which support for mobile (wireless) devices such as cell phones and personal data assistants (PDAs).
- o Security changes - enable Windows Forms assemblies to execute in a semi-trusted manner from the Internet, and enable **Code Access Security** in ASP.NET applications.
- o Built-in support for ODBC and Oracle databases. The .NET Framework Data Provider for ODBC, which previously was available as a Web download only, now ships with the .NET Framework under the namespace **System.Data.Odbc** for ODBC, and for Oracle now ships with the .NET Framework under the namespace **System.Data.OracleClient**.
- o The .NET Framework version 1.1 supports **side-by-side execution**. Side-by-side execution is the ability to store and execute multiple versions of an application or component on the same computer. It means subsequent installations of other versions of the .NET Framework or of a component will not affect the applications already installed.
- o Internet Protocol version 6 (IPv6) support.

(c) .NET Framework 2.0

The .NET Framework version 2.0 improves scalability and performance of applications with improved caching, application deployment. It introduced a new version of the CLR with additions to the base class libraries, including generics, generic collections, and significant additions to ASP.NET.

• Changes in .NET Framework 2.0 in comparison with .NET Framework 1.1

- o It provides Support for 64-bit applications that enable users to build managed code libraries or easily use unmanaged code libraries on 64-bit versions of windows.

- o An access control list (ACL) is used to grant or revoke permission to access a resource on a computer. New classes have been added to the .NET Framework that allow managed code to create and modify an ACL. New members that use an ACL have been added to the I/O, registry, and threading classes.
- o New features in ADO.NET include support for user-defined types (UDT), asynchronous database operations, XML data types, large value types, snapshot isolation, and new attributes that allow applications to support multiple active result sets (MARS) with SQL Server 2005.
- o Many additional and improved ASP.NET web controls which make it easier to add commonly used functionality to dynamic Web pages. It also make possible to display and edit data on an ASP.NET Web page without writing code.
- o New personalization features for ASP.NET, such as support for themes, skins, master pages and web parts.
- o The new Data Protection API (DPAPI) includes four methods that allow applications to encrypt passwords, keys, and connections strings.
- o Provides FTP support through which applications can access File Transfer Protocol resources using the **WebRequest**, **WebResponse**, and **WebClient** classes.
- o .Net Framework 2.0 supports nullable types, which means we can assign either normal values to the variables or a null value. For example, we can set value of an integer variable as **null**.

(d) .NET Framework 3.0

.NET Framework 3.0 was designed to be shipped as an integral part of Windows Vista Operating System and Windows Server 2008. Nevertheless, .NET 3.0 also works fine with Windows XP and Windows Server 2003.

• Changes in .NET Framework 3.0 in comparison with .NET Framework 2.0

- o Development Time is reduced.
- o This version is essentially .NET Framework 2.0 with the addition of:
 - ✓ **Windows Presentation Foundation (WPF):** It provides tools to create graphically enhanced documents and web pages. It uses 3D graphics hardware and Direct3D technologies.
 - ✓ **Windows Communications Foundation (WCF):** It is a service-oriented messaging system which allows programs to interoperate locally or remotely. It is similar to Web Services.
 - ✓ **Windows Workflow Foundation (WF):** The purpose of WF is to let developers create and execute workflow-based applications. It represents the application activities graphically like flowcharts and finite state machines.
 - ✓ **Windows CardSpace:** It is a software component which securely stores a person's digital identities and provides a unified interface for choosing the identity for a particular transaction, such as logging in to a website.

(e) .NET Framework 3.5

Version 3.5 SP1 of the .NET Framework shipped with Windows 7. It also shipped with Windows Server 2008 R2 as an optional component.

- **Changes in .NET Framework 3.5 in comparison with .NET Framework 3.0**
 - o .NET Framework 3.5, Added some new features such as AJAX-enabled Web sites and support for LINQ (XML, Objects and Dataset).
 - o BCL enhancements: Compression support, Client-side certificates, Sound APIs.
 - o The SP1 update added the .NET Framework Client Profile, Dynamic Data, and a small set of additional enhancements.
 - o .NET Framework 3.5 provide support for additional features on Windows Mobile and Windows Embedded CE devices.
 - o This release adds new functionality and provides performance improvements under certain conditions, especially with WPF where 20–45% improvements are expected.
 - o Two new data service components have been added:
 - ✓ ADO.NET Entity Framework.
 - ✓ ADO.NET Data Services.
 - o Two new assemblies for web development have been added:
 - ✓ System.Web.Abstractions
 - ✓ System.Web.Routing
 - o It also featured a new set of controls called "Visual Basic Power Packs" which brought back Visual Basic controls such as "Line" and "Shape".

(f) .NET Framework 4.0

It includes a new version of the CLR, expanded base class libraries, and new features such as the Managed Extensibility Framework (MEF), dynamic language runtime (DLR), and code contracts.

- **Changes in .NET Framework 4.0 in comparison with .NET Framework 3.5**
 - o Improvements in ADO.NET.
 - o It introduces an improved security model.
 - o New namespaces, types, and members added to existing types.
 - o The .NET Framework 4 is highly compatible with applications that are built with earlier .NET Framework versions, except for some changes that were made to improve security, standards compliance, correctness, reliability, and performance.
 - o Provides support for parallel computing, which target multi-core or distributed systems.
 - o It also includes Parallel Language Integrated Query (PLINQ).
 - o New Visual Basic .NET and C# language features, such as
 - Implicit line continuations.

- Dynamic dispatch.
- Named parameters, and
- Optional parameters.
- o It also includes new types to work with arbitrary-precision, such types are.
 - System.Numerics.BigInteger for arithmetic.
 - System.Numerics.Complex for complex numbers.

(g) .NET Framework 4.5 (Upcoming)

The .NET Framework 4.5 Developer Preview is only supported on Windows 7 or later.

The key focus is on:

- o Better performance when retrieving resources.
- o Zip compression improvements to reduce the size of a compressed file.
- o Managed Extensibility Framework (MEF) that support for generic types.
- o New Asynchronous File Operations features were added to the C# and Visual Basic languages. These features add a task-based model for performing asynchronous operations.
- o Improvements in ASP.NET
 - ✓ Support for model binders in Web Forms. These let you bind data controls directly to data-access methods, and automatically convert user input to and from .NET Framework data types.
 - ✓ Support for JavaScript in client-side validation scripts.
 - ✓ Improved handling of client script for improved page performance.
 - ✓ Support for WebSockets protocol.
 - ✓ Support for reading and writing HTTP requests and responses asynchronously.
- o Support for Internationalized Domain Name (IDN) parsing.
- o Support for Email Address Internationalization (EAI).

Understanding the .NET Framework is of primary importance in developing applications with Visual Basic 2010. The .NET Framework family also includes two versions for mobile or embedded device use. A reduced version of the framework, the .NET Compact Framework, is available on Windows CE platforms, including Windows Mobile devices such as smartphones. Additionally, the .NET Micro Framework is targeted at severely resource-constrained devices.

1.8 MICROSOFT INTERMEDIATE LANGUAGE

An intermediate language generated by compiler is called MSIL. We can call it as Intermediate Language (IL) or Common Intermediate Language (CIL). During the compile time, the compiler convert the source code into Microsoft Intermediate Language (MSIL)

Microsoft Intermediate Language (MSIL) is a CPU-independent set of instructions that can be efficiently converted to the native code. MSIL code is not executable but further processed by CLR before it becomes executable. During the runtime the Common Language Runtime (CLR)'s Just In Time (JIT) compiler converts the Microsoft Intermediate Language (MSIL) code into native code to the Operating System. When a compiler produces Microsoft Intermediate Language (MSIL), it also produces Metadata. The Microsoft Intermediate Language (MSIL) and Metadata are contained in a portable executable (PE) file. MSIL includes instructions for:

- o Loading, storing, initializing, and calling methods on objects.
- o Arithmetic and logical operations.
- o Control flow.
- o Direct memory access.
- o Exception handling and other operations.

Figure 1.7 shows the how MSIL (IL or CIL) helps to execute a .NET application.

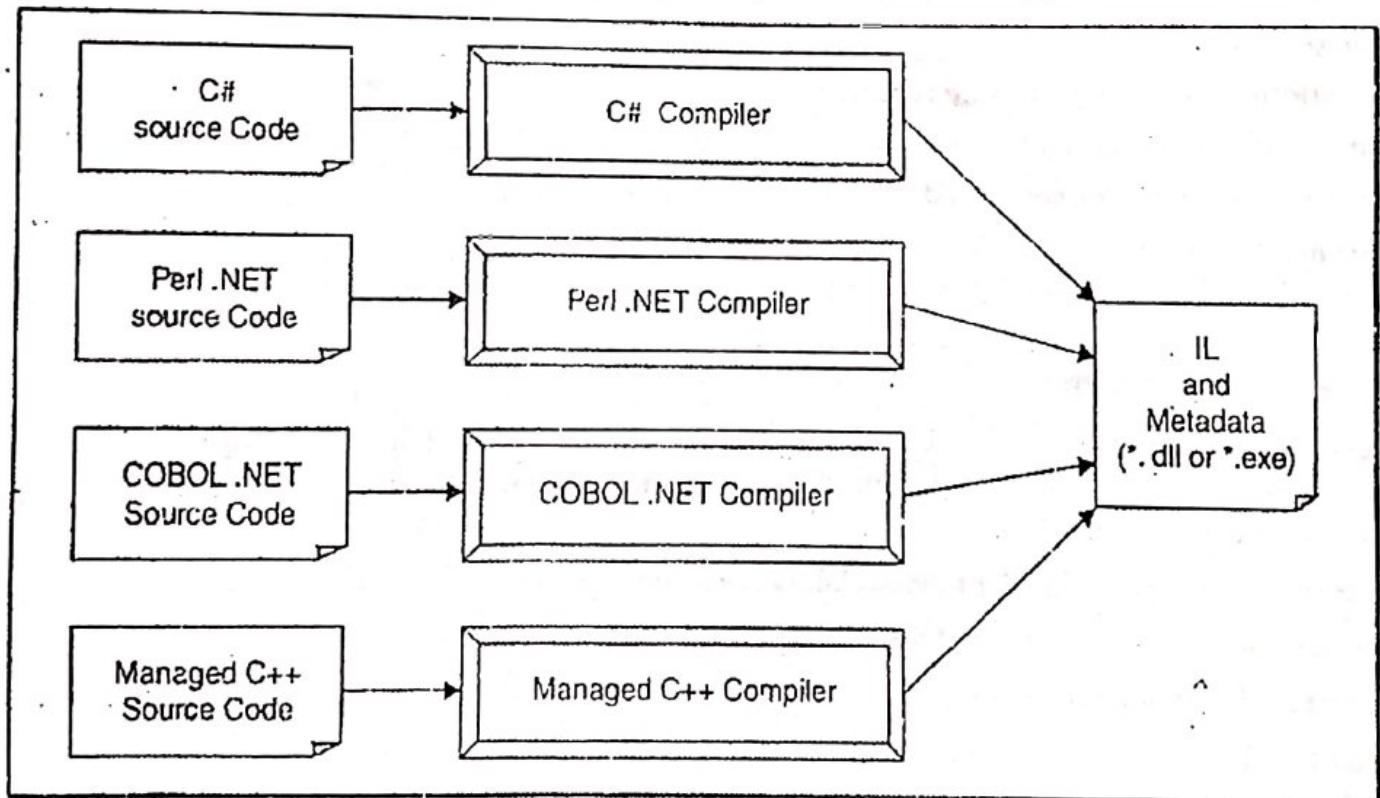


Fig. 1.7

All .NET source codes written in any .net supportive language (C#, VB.net etc), when compiler are converted to MSIL. This MSIL, at the Runtime, gets converted to machine code. The Runtime convention of MSIL code to the machine code is handled by a component called as the Just In Time (JIT) Compiler. In other words, when we compile the code that uses the .NET framework library we don't immediately create operating system specific code, we compile our code into IL. This IL is not specific to any O/S and also is not specific to any language supported by the .NET languages e.g. VB.NET, C#, J#, VC++ etc. IL is also a language, which is converted into machine code JIT (just in time) compiler. IL is independent of the O/S machine.

1.9 META DATA

The information about the code such as the types and references that the code contains are required at runtime to execute the code. This information is in the **metadata**. Metadata is simply a set of data tables that describe what is defined in the module, such as types and their members. Metadata is used to describe an Assembly. It describes its **Identity**, **expected types**, **dependencies** in other assemblies, and **required security permissions**.

A .NET language compiler will generate the metadata and store this in the assembly containing the CIL. When the CLR executes CIL it will check to make sure that the metadata of the called method is the same as the metadata that is stored in the calling method. This ensures that a method can only be called with exactly the right number of parameters and exactly the right parameter types. Metadata describes every Type by specifying information such as

- o Its name, visibility, base class, and the interface it implements.
- o Type members such as **methods**, **fields**, **properties**, etc.]

Metadata also has tables indicating what the managed module references, such as imported types and their members. This description is independent of any programming language. It's not in C# or IL. Metadata is specified in a language-neutral format. The MSIL code and the metadata is located in Portable Executable file. When this Portable Executable (PE) file is executed, the class loader component of the CLR loads the MSIL code and the metadata from the Portable Executable file into the run time memory.

1.9.1 Benefits/Uses of Metadata

- Metadata removes the need for header and library files while compilation, because all the information about the referenced types/members is contained in one file along with the MSIL that implements those types/members. Compilers can read metadata directly from managed modules. *Reference & Data addn*
- Metadata help you to write code. Its IntelliSense feature parses metadata to tell you, what methods a type offers and what parameters that method expects, *while parameters option for*
- Metadata allows the garbage collector to track the lifetime of objects. For any object, the garbage collector can determine the type of the object, and from the metadata it knows which fields within that object refer to other objects. *delete obj by garbage collector*
- The CLR code verification process uses metadata to ensure that your code performs only "safe" operations.

1.10 .NET TYPES

Types are the building blocks of programs. Type can be something simple, like an integer, or it can be something more complex, like an Employee structure representing an employee. All types derive from the System. Object base type. There are two general categories of types, each of which is further divided into subcategories:

- o Value types.
- o Reference types.

(a) Value types

Value types directly contain their data. Instances (objects) of value types are either allocated on the **stack** or allocated inline in a structure. Value types can be built-in (implemented by the runtime), user-defined, or enumerations. Variables that are of value types, each have their own copy of the data, and therefore operations on one variable do not affect other variables. A value type is copied on assignment, so if we assign one value type to another, the actual underlying data is copied.

Example1: if we have an employee structure and we assign one employee to another, the actual employee data such as first name, last name and so on is copied to another one. They have no size or memory management overheads.

Example2: if you create a value type which has data that takes 4 bytes, then instance of it will consume 4 bytes. And memory management is not an issue because a value type is allocated on the stack so its memory is reclaimed automatically.

Note: Value types can include a structure, an enumeration and most of the simple types like int, boolean etc.

(b) Reference types

Reference types store a reference to the value's memory address. They are allocated on the **heap**. They can be self-describing types, pointer types, or interface types. The type of a reference type can be determined from values of self-describing types. Self-describing types are further split into arrays and class types. The class types are user-defined classes, boxed value types, and delegates. They can refer to the same object. Therefore, operations on one variable can affect the same object referred to by another variable. When assigning a reference type, only the reference is copied.

Example: Assume that employee is defined as a reference type and we have a reference to an instance of employee. Now if assign this employee to another employee, we end up with two references that are referring to the same employee instance.

There are no two employee instances, just one instance with two references to it. Reference types have memory overhead of 8 to 16 bytes depending on whether the program is running on a 32 bit or 64 bit operating system. They are also subject to garbage collection. When something is located in the heap even when it's no longer in use and until garbage collection occurs and claims the memory.

The following example shows the difference between reference types and value types.

```
using System;  
class Class1
```

```
{  
    public int Value = 0;  
}
```



```

class Test
{
    static void Main() {
        int val1 = 0;
        int val2 = val1;
        val2 = 123;
        Class1 ref1 = new Class1();
        Class1 ref2 = ref1;
        ref2.Value = 123;
        Console.WriteLine("Values: {0}, {1}", val1, val2);
        Console.WriteLine("Refs: {0}, {1}", ref1.Value, ref2.Value);
    }
}

```

Output:

Values: 0, 123

Refs: 123, 123

Fig. 1.8 diagram illustrates how these various types are related.

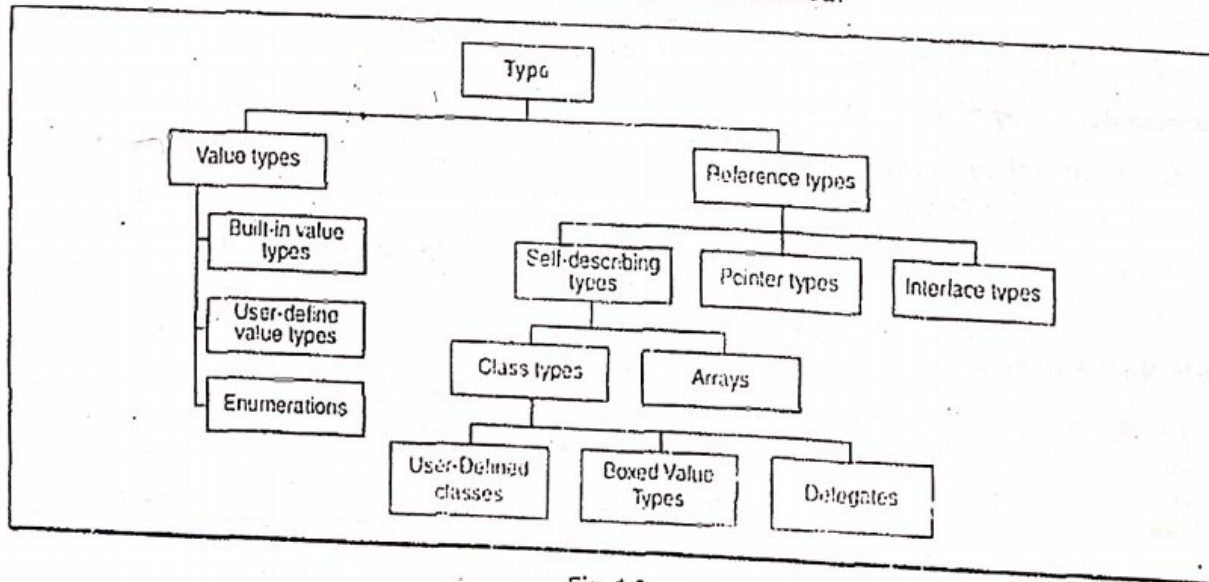


Fig. 1.8

We can define new types from existing types. Types can be nested, that is, a type can be a member of another type.

A type definition includes:

- Any attributes defined on the type.
- The type's visibility.
- The type's name.
- The type's base type.

- Any interfaces implemented by the type.
- Definitions for each of the type's members.

1.11 .NET NAMESPACES

A library of related classes in .NET is called as a Namespace. The Framework Class Library (FCL) of .NET contains thousands of types, a set of related types is presented to the developer within a single namespace.

For Example: The **System** namespace contains the **Object** base type, from which all other types ultimately derive. In addition, the **System** namespace contains:

- o Types for integers, characters, strings, exception handling, and console I/O.
- o As well as a bunch of utility types that convert safely between data types, format data types, generate random numbers, and perform various math functions.

All applications use types from the **System** namespace. To access any platform feature, you need to know which namespace contains the types that provide the facility you want. .NET also enables developers to create their own namespaces containing their own types. There are following main points regarding namespace:

- A namespace is a collection of different classes.
- All applications are developed using classes from the .NET System namespace.
- The namespace with all the built-in functionality is the **System** namespace. All other namespaces are based on this **System** namespace.
- From the viewpoint of the runtime, a namespace is just a collection of type names.
- Namespaces provides a way of organizing related classes and other types.
- Namespaces also reduce the conflict of use of same class name, because we can use fully qualified name instead of using namespace plus the class name both are same.

The table below lists some of the standardized and non-standardized namespaces, with a brief description of what the types in that namespace are used for:

(a) Standardized Namespaces

These are the namespaces that are standardized as of the ECMA 335 and ISO/IEC 23271:2006 standards.

Namespace	Purpose of Types
System	• It includes all the basic types used by every application such as String, DateTime, Boolean, etc. • Support for environments such as the console. • Support for Math functions, and base classes for attributes, exceptions, and arrays.

System.Collections	- Managing collections of objects. - Includes the popular collection types such as Stacks, Queues, Hash tables, etc.
System.Diagnostics	- Provides the ability to diagnose applications. - It includes event logging, performance counters, tracing, and interaction with system processes.
System.Globalization	It provides information such as calendars in use, format patterns for date, currency and numbers.
System.IO	- Enables reading from and writing to different streams, such as files or other data streams. - Also provides a connection to the file system.
System.Net	- It is used for Network communications. - It provides an interface for many of the protocols used on networks, such as HTTP, FTP, and SMTP.
System.Reflection	It Inspects metadata and late binding to types and their members.
System.Runtime.InteropServices	- Enabling managed code to access unmanaged OS platform facilities, such as COM components and functions in Win32 DLLs. - Provide Interoperability with COM or other native code.
System.Runtime.Remoting	Accessing types remotely.
System.Runtime.Serialization	Enabling instances of objects to be persisted and regenerated from a stream.
System.Security	- It helps to protect data and resources. - This namespace allows security to be built into applications based on policy and permissions. - It provides services such as cryptography.
System.Text	- It supports various encodings like ASCII or Unicode. - It also supports regular expressions, and a more efficient mechanism for manipulating strings.
System.Threading	- It facilitates multithreaded programming. - It allows the synchronizing of "thread activities and access to data." - It also synchronizing access to resources.
System.Xml	- Provides support for processing XML schemas and data. - Including reading, writing, schemas, serialization, searching, and transforming.

b) Non-Standardized Namespaces

These are the namespaces that are not standardized as of the ECMA and/or ISO standards, and are specific to Microsoft implementation. However, even if implementing them is not mandatory, some of them may have been implemented completely or partially by other .NET implementations.

Namespace	Purpose of Types
System.CodeDom	This library provides the ability to create code and run it, at runtime.
System.ComponentModel	- Provides the ability to implement the run-time and design-time behavior of components and controls. - It contains the infrastructure for implementing attributes and type converters, binding to data sources, and licensing components.
System.Configuration	Provides the infrastructure for handling configuration data.
System.Data	It represents the ADO.NET architecture, which is a set of computer software components that can be used by programmers to access data and data services.
System.Deployment	It allows customization of the applications upgrade when using ClickOnce.
System.DirectoryServices	It provides easy access to Active Directory from managed code.
System.Drawing	- It is used to manipulate 2D and vector graphics. - It provides support for imaging, printing, and text services. - It is used for Windows Forms applications.
System.EnterpriseServices	It is used to manage transactions, queued components, object pooling, just-in-time activation, security, and other features to make the use of managed code more efficient on the server.
System.Linq	It defines interfaces and methods, that lets LINQ providers to be plugged in.
System.Linq.Expressions	It allows Delegates and expressions to be represented as expression trees, so that the high-level code can be examined and processed at runtime.
System.Management	- It allows querying for system information, such as: - How much free space is left on the disk.

	- o What is the current CPU utilization. - o Which database a certain application is connected to, and much more.
System.Media	• It provides the ability to play system sounds and .wav files.
System.Messaging	• It provides the ability to connect to, monitor, and administer message queues on the network and send, receive, or peek messages. • It also named as .NET Remoting. • This namespace is being superseded by Windows Communication Foundation.
System.Resources	• It is used to Manipulate and managing external data resources.
System.ServiceProcess	• It allows the creation of applications that run as a service within Windows.
System.Timers	• It Allows you to raise an event on a specified interval.
System.Transactions	• It provides support for local or distributed transactions.
System.Web	• It provides various web related functionality. • It enables browser-server communication and the creating XML Web services. • Most or all of these libraries are referred to as the ASP.NET architecture.
System.Windows.Forms	• It is used to build Windows GUI applications. • This allows for writing graphical applications for Windows from within managed code. • This system is being superseded by the Windows Presentation Foundation.

1.12 COMMON LANGUAGE RUNTIME

Common Language Runtime (CLR) is the engine available in .Net Framework which provides the runtime environment and various runtime services to the applications. The CLR locates, loads and executes our programs.

The Common Language Runtime consists of components that

- o Loads the IL code of a program into the runtime,
- o Compile the IL code into native code,
- o Execute and manage the code,
- o Enforce security and type safety, and provide thread support and other useful services.

The code that runs within the CLR is called as managed code, whereas the code that runs outside the CLR is called unmanaged code. The CLR also provides Interoperability layer, which allows both the managed and unmanaged code to interoperate with each other.

Components of CLR

Various components of the CLR work together to provide the runtime infrastructure to the applications that run on the .NET Framework. The components of CLR provide an exception manager, a common debugger and other system services such as base class library support and interoperability. These components include:

- o **Class Loader:** It loads classes into the runtime.
- o **MSIL to Native Code Compiler:** It converts MSIL code into native code.
- o **Code Manager:** It manages the code during execution.
- o **Garbage Collector:** It performs automatic memory management.
- o **Security Engine:** It enforces security restrictions.
- o **Type Checker:** It enforces strict type checking.
- o **Thread Support:** It provides multithreading support to applications.
- o **Exception Manager:** It provides a mechanism to handle the exceptions.
- o **Debug Engine:** It allows you to debug different kinds of applications.
- o **COM Marshaler:** It allows .NET applications to exchange data with COM applications.
- o **Base Class Library Support:** It provides the types that the applications need at run time.

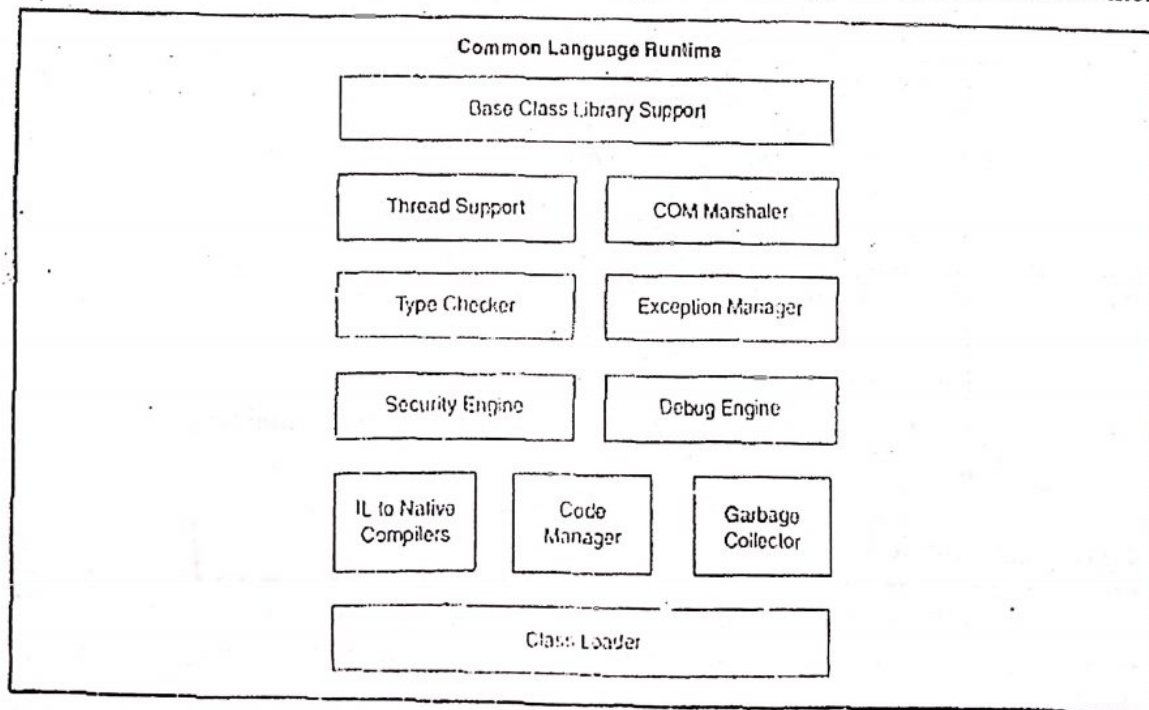


Fig 1.9: Components of the Common Language runtime/ Architecture of CLR

The steps

(a) First,

(b) The

comp

(c) After

code.

(d) The M

load,

a.

b.

c.

d.

(e) To ex

that t

(f) The M

(g) When

CLR l

time i

(h) Before

(i) The II

archit

(j) The II

code c

CLS-
Lo

v

J

Oil
Lar

How CLR Works

The steps followed during CLR are

- (a) First, the program is to be written in any CLS-compliant Language.
- (b) The source code of the program is first compiled by the CLS-compliant language compiler.
- (c) After compilation Intermediate Language code is generated which is known as MSIL code.
- (d) The MSIL code contains a CPU- independent set of instructions, which describes how to load, store, initialize, and call methods on objects. It also contains:
 - a. instructions that enable you to perform arithmetic and logical operations.
 - b. Access memory directly.
 - c. Control the flow of execution.
 - d. Handle exceptions, and perform other operations.
- (e) To execute the code, the information about the code such as the types and references that the code contains are required at runtime. This information is in the metadata.
- (f) The MSIL code and the metadata is located in Portable Executable file.
- (g) When this Portable Executable (PE) file is executed; the class loader component of the CLR loads the MSIL code and the metadata from the Portable Executable file into the run time memory.
- (h) Before the code is executed, it is passed to the IL to native code compiler for compilation.
- (i) The IL to Native code compiler contains Just In Time (JIT) compilers for different CPU architectures and compiles IL code into a native instruction set or machine code.
- (j) The IL to native code compilation required to be done only once. After that the machine code can be executed any number of time.

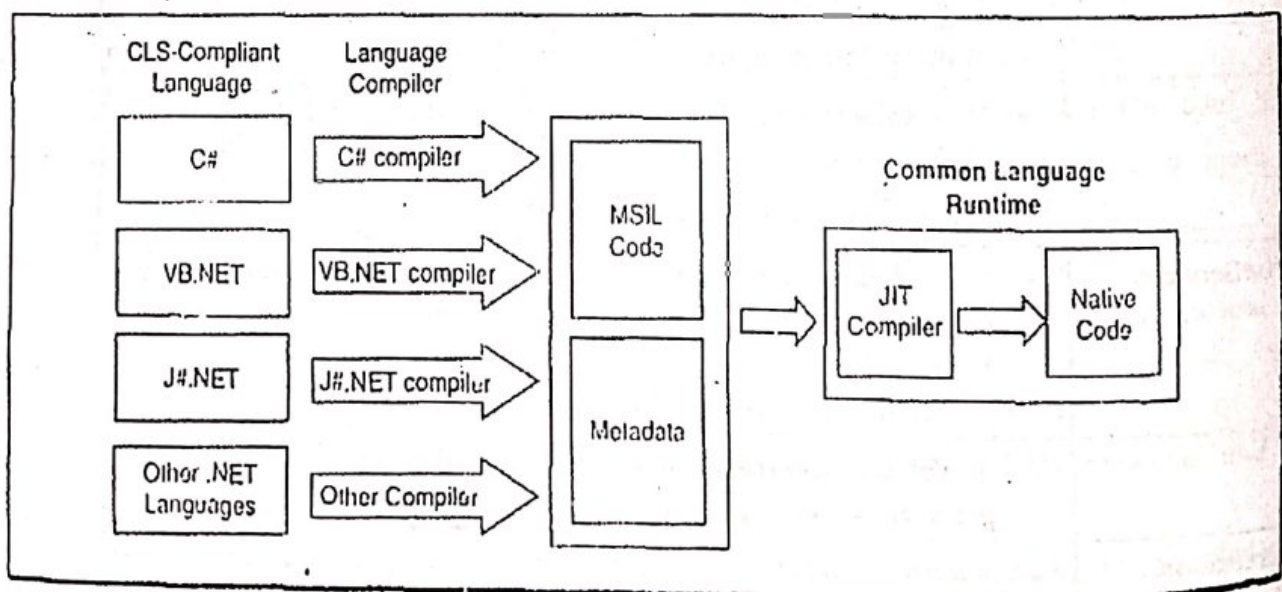


Fig 1.1C: Execution of a .NET Application

Services provided by CLR

The CLR provides a run-time environment that manages the execution of code and provides services that improve development, deployment, and run time. Such are as follows:

- o Code management.
- o Automatic Memory management.
- o Verification of type safety.
- o Conversion of MSIL to native code.
- o Loading and execution of managed code.
- o Creation and management of metadata.
- o Insertion and execution of security checks.
- o Handling cross-language exceptions.
- o Automation of object layout for late binding.
- o Developer services (profiling, debugging, etc.).

1.13 COMMON TYPE SYSTEM

The types in .NET Framework are the base on which .NET applications, components, and controls are built. Common Type System (CTS) in .Net Framework defines how types are going to be declared, used and managed in runtime. It includes:

- o What kinds of types are available such as Value types and Reference types.
- o What these types contain such as Fields, Methods, Properties and Events.
- o How they are visible to other types such as public, private, etc.

In other words, it provides the necessary data types, value and object types, which you need to develop applications in different languages. All the .NET languages share CTS. This implies that a String in Visual Basic .NET is the same as a String in Visual C# or in Visual C++ .NET. All the .NET languages have access to the same class libraries.

In addition, all languages are equally powerful. There is no superior language in .NET i.e. Visual Basic .NET is as powerful as Visual C# or Visual C++ .NET. In .Net Framework, the System namespace is the root for all the data types. This namespace consists of classes such as Object, Byte, String, and Int32 that represents base data types. These base data types are used by all applications.

For Example, An integer variable in C# is written as int, whereas in Visual Basic it is written as Integer. Therefore in .Net Framework you have single class called System.Int32 to interpret these variables.

There are following functions performed by CTS:

- o Automatically adapts itself in a framework that enables integration of multiple languages, type safety, and high performance code execution.
- o Provides an object-oriented model.
- o Standardizes the conventions that all the languages must follow.
- o Invokes security checks.
- o Encapsulates data structures.

1.14 COMMON LANGUAGE SPECIFICATIONS

Common language Specifications (CLS) allows objects created in different languages to communicate with one another. It integrates all languages to let objects created in one language be treated as equal citizens by code written in a completely different language. To make this possible, the CLR defines a standard behavior for types, embeds self-describing type information (metadata), and provides a common execution environment. *changed for sake*

Language integration is a fantastic goal, of course, but the truth of the matter is that programming languages are very different from one another. For example, some languages lack features commonly used in other languages:

- o Case-sensitivity.
- o Unsigned integers.
- o Operator overloading.
- o Methods that support a variable number of parameters.

method etc. for converted and change

If you intend to create types that are easily accessible from other programming languages, then it is important that you use only those features of your programming language that are guaranteed to be available in all other languages.

To help you with this, Microsoft has defined a Common Language Specification. (CLS) that informs compiler vendors of the minimum set of features that their compilers must support to target the common language runtime. Several vendors are already planning .NET-compatible versions of their compilers.

Note that the CLR/CTS supports a lot more features than the subset defined by the common language specification, so if you don't care about language interoperability, you can develop very rich types limited only by the capabilities of that particular language.

The figure below summarizes the way in which language features overlap with the CLS, within the wider context of the CLR/CTS.

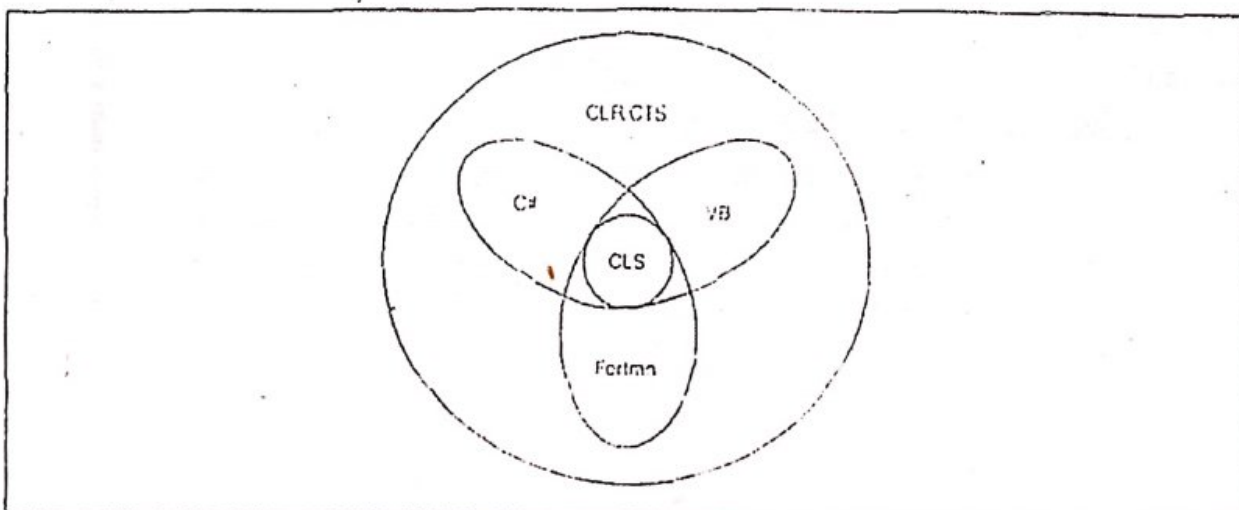


Fig. 1.11

As this figure 1.11 shows, the CLR/CTS offers a broadly inclusive set of features.

- Some languages expose a large subset of the CLR/CTS. In fact, a programmer willing to write in IL assembly language is able to use all the features offered by the CLR/CTS.
- Most other languages—such as C#, VB, and Fortran—expose a subset of the CLR/CTS features to the programmer.
- The CLS defines a minimum set of features that all languages must support.

If you are designing a type in one language and you expect that type to be used by another language, then you should not take advantage of any features that are outside of the CLS. Doing so means that your type members might not be accessible by programmers writing code in other programming languages.

1.15 .NET APPLICATIONS USING COMMAND LINE COMPILER AND VISUAL STUDIO .NET IDE

.NET Framework is a set of Framework Class Library (FCL) assemblies that contains several thousands of type definitions, where each type exposes some functionality. All in all, the CLR and the FCL allow developers to build the following kinds of applications:

- Windows Console Applications
- Windows GUI applications (Windows Forms)
- Web Services
- Web Forms
- Windows Services
- Component Library

All the above applications are explained below:

• Windows Console Applications

A Console application is an application which is designed to be used by text-based interfaces only. A user can interact with a console application using only a keyboard and display screen. The generation of console applications is kept as a feature of modern programming environments such as Visual Studio and the .NET Framework on Microsoft Windows because it greatly simplifies the learning process of a new programming language by removing the complexity of graphical user interfaces. Console applications are the applications with very simple UI demands. A console application provides a quick and easy solution. Compilers, utilities, and tools are typically implemented as console applications.

• Windows GUI applications (Windows Forms)

Windows GUI is a powerful and high-performance functionality offered by the Windows Forms applications. Windows forms applications can take advantage of controls, menus, mouse and keyboard events, and can talk directly to the underlying operating system. Windows form applications provides Integrated Development Environment (IDE) for GUI applications. These applications are easy to use and understand.

- **Web Services**

web page

Web services are the components that can be accessed over the Internet very easily. These are core to application integration in the .NET environment. It allow internet as well as intranet applications, written in different languages and hosted on different platforms, to exchange data using standard protocols such as HTTP, XML and SOAP. We can say that a Web Service is an application that is designed to interact directly with other applications over the internet.

A Web Service is Language Independent, Protocol Independent, and Platform Independent. It assumes a stateless service architecture.

- **Web Forms**

A Web Form is a Web Page. A Web Form allows a user to enter data that is sent to a server for processing. A Web form application can take advantage of controls, menus, mouse and keyboard events. These Web Forms can be defined as HTML-based applications (**web sites**). Typically, Web form applications make database queries and web service calls, combine and filter the returned information, and then present that information in a browser using a rich HTML-based UI. Web form provides an Integrated Development Environment (IDE) for web applications.

- **Windows Services**

A Windows service is a long-running executable that performs specific functions. Working with services can feel like configuring some secret part of our PC, but actually services are just programs. There are two ways that services usually differ from regular programs:

Firstly, services require no user intervention to run. A service would not pop up any windows asking for our input, instead it will simply carry out its tasks silently in the background.

Automatic
In simple words, Windows services can be configured to start when the operating system is booted and run in the background as long as Windows is running, or they can be started manually when required.

Secondly, services are managed by the Service Control Manager interface provided by Windows, where they can be enabled and disabled.

There are many services are provided by Microsoft. Some services must be started when Windows starts up to enable normal operation of your computer. Other services are installed by third parties when you install their software.

- **Component Library**

The .NET Framework allows you to build stand-alone components (types) that may be easily incorporated into any of the above mentioned application types.

REVIEW QUESTIONS

1. What is .NET and .NET Framework?
2. What is .NET technology? Describe briefly its origins.
3. List the tools provided by .NET Framework for managing the user and application interfaces.

4. What is Web service? How is it achieved using the .NET strategy?
5. What is the Microsoft Intermediate Language (MSIL)?
6. What is the Common Language Runtime (CLR)?
7. Enumerate major services provided by the CLR.
8. What are the additional benefits provided by CLR to programs developed using C#?
9. How the CLR does implements a C# program?
10. What is the Common Type System?
11. What is the Common Language Specification?
12. What is managed code?
13. List some of the important services the Framework Base Classes can offer to the users.
14. Describe the application of Visual Studio .NET?
15. List the languages supported by the .NET Framework.
16. What are the benefits of .NET strategy advanced by Microsoft?

